DBMS ASSIGNMENT SERIES

Assignment 8

Basic SQL Queries and Aggregate Functions

Online Food Delivery System

SUBMITTED BY	PROGRAMME & INSTITUTION	SUBMISSION DATE
Vimal Kumar Rao	BCA (AI/ML) Alliance University	24 October 2026

A consistent relational design carried forward from Assignments 1 and 2.

IN THIS ASSIGNMENT

01	02	03	04
Filter rows	Sort results	Group & aggregate	Explain output

1. Objective

This assignment demonstrates filtering, sorting, grouping, grouped filtering, and aggregate calculations using the Online Food Delivery System. The query file contains eight executable MySQL SELECT statements; output tables below show the corresponding results from the bundled fictional dataset.

> **Output note:** These result tables were executed against the same sample rows in a local SQLite validation harness because a MySQL server was not available in the authoring environment. The SELECT statements are MySQL-compatible. The query_outputs folder contains table snapshots for review.

Q1. WHERE - Delivered orders

Filters the order headers to rows whose status is Delivered, then sorts newest first. WHERE filters individual records before rows are returned.

```
SELECT Order_ID, Order_Date, Order_Status
FROM Orders
WHERE Order_Status = 'Delivered'
ORDER BY Order_Date DESC;
```

Order_ID	Order_Date	Order_Status
8	2026-10-24 08:45:00	Delivered
4	2026-10-22 20:00:00	Delivered
3	2026-10-22 13:20:00	Delivered
2	2026-10-21 19:05:00	Delivered
1	2026-10-21 12:15:00	Delivered
11	2026-10-20 20:00:00	Delivered

Q2. ORDER BY - Most expensive menu items

Sorts menu items by price descending; Item_Name provides a stable tie-breaker. LIMIT returns the five highest-priced rows.

```
SELECT Item_ID, Item_Name, Price
FROM Menu_Item
ORDER BY Price DESC, Item_Name ASC
LIMIT 5;
```

Item_ID	Item_Name	Price
8	Farmhouse Pizza	329
7	Margherita Pizza	249
1	Chicken Biryani	220
14	Chilli Paneer	220
2	Paneer Biryani	180

Q3. GROUP BY - Orders by restaurant

Counts the number of order headers per restaurant. LEFT JOIN keeps restaurants even if they have zero orders.

```
SELECT r.Restaurant_Name, COUNT(o.Order_ID) AS Total_Orders
FROM Restaurant r
LEFT JOIN Orders o ON o.Restaurant_ID = r.Restaurant_ID
GROUP BY r.Restaurant_ID, r.Restaurant_Name
ORDER BY Total_Orders DESC, r.Restaurant_Name ASC;
```

Restaurant_Name	Total_Orders
Spice Route Kitchen	3
Urban Pizza Co.	3
Dosa Junction	2
Green Bowl Cafe	2
Wok Express	2

Q4. HAVING - Repeat customers

Groups orders by customer and keeps groups with at least two orders. HAVING filters aggregate groups; WHERE would filter rows before grouping.

```
SELECT c.Name, COUNT(o.Order_ID) AS Order_Count
FROM Customer c
JOIN Orders o ON o.Customer_ID = c.Customer_ID
GROUP BY c.Customer_ID, c.Name
HAVING COUNT(o.Order_ID) >= 2
ORDER BY Order_Count DESC, c.Name ASC;
```

Name	Order_Count
Arjun Reddy	2
Asha Menon	2
Meera Iyer	2
Priya Sharma	2
Rahul Nair	2

Q5. Aggregate functions - COUNT, SUM, AVG, MIN, MAX

Calculates the number of order lines, total line-item value, average unit price, lowest unit price, and highest unit price. Cancelled orders are excluded from the valuation.

```
SELECT COUNT(*) AS Item_Lines,
    SUM(oi.Quantity * oi.Unit_Price) AS Line_Value_Total,
    ROUND(AVG(oi.Unit_Price), 2) AS Average_Unit_Price,
    MIN(oi.Unit_Price) AS Lowest_Unit_Price,
    MAX(oi.Unit_Price) AS Highest_Unit_Price
FROM Order_Item oi
JOIN Orders o ON o.Order_ID = oi.Order_ID
WHERE o.Order_Status <> 'Cancelled';
```

Item_Lines	Line_Value_Total	Average_Unit_Price	Lowest_Unit_Price	Highest_Unit_Price
20	4225	162.30	40	329

Q6. GROUP BY and SUM - Subtotal by order status

Summarizes order count and line-item subtotal for each order status. Cancelled order totals are displayed but should not be treated as sales.

```
SELECT o.Order_Status,
    COUNT(DISTINCT o.Order_ID) AS Orders,
    SUM(oi.Quantity * oi.Unit_Price) AS Line_Subtotal
FROM Orders o
JOIN Order_Item oi ON oi.Order_ID = o.Order_ID
GROUP BY o.Order_Status
ORDER BY Orders DESC, o.Order_Status ASC;
```

Order_Status	Orders	Line_Subtotal
Delivered	6	2196
Placed	2	729
Cancelled	1	150
Confirmed	1	420
Out for Delivery	1	480
Preparing	1	400

Q7. GROUP BY date - Daily subtotal

Aggregates non-cancelled order lines by calendar day to show daily demand and line-item value.

```
SELECT DATE(o.Order_Date) AS Order_Day,
    COUNT(DISTINCT o.Order_ID) AS Orders,
    SUM(oi.Quantity * oi.Unit_Price) AS Line_Subtotal
FROM Orders o
JOIN Order_Item oi ON oi.Order_ID = o.Order_ID
WHERE o.Order_Status <> 'Cancelled'
GROUP BY DATE(o.Order_Date)
ORDER BY Order_Day ASC;
```

Order_Day	Orders	Line_Subtotal
2026-10-20	1	498
2026-10-21	2	890
2026-10-22	2	628
2026-10-23	3	1209
2026-10-24	3	1000

Q8. GROUP BY - Payment status distribution

Counts payment records by status, helping identify pending and failed payment follow-up.

```
SELECT Payment_Status, COUNT(*) AS Payment_Records
FROM Payment
GROUP BY Payment_Status
ORDER BY Payment_Records DESC, Payment_Status ASC;
```

Payment_Status	Payment_Records
Paid	6
Pending	4
Failed	1
Refunded	1

2. Interpretation

- WHERE and ORDER BY answer row-level questions such as which orders are delivered and which menu items are most expensive.
- GROUP BY plus COUNT shows workload by restaurant and payment state; HAVING isolates customers with repeated orders.
- COUNT, SUM, AVG, MIN, and MAX summarize order-line data. Aggregating after joining order lines can multiply order-header values, so use COUNT(DISTINCT Order_ID) when counting orders across line items.
- The stored Unit_Price is the historical unit price, so line subtotal is computed as Quantity × Unit_Price rather than current Menu_Item.Price.
- Cancelled orders are excluded from the daily and non-cancelled subtotal calculations where stated. Business reports must define whether pending or refunded orders count as revenue.

3. Execution

Open Assignment_8_Basic_SQL_Queries.sql in MySQL Workbench after running Assignments 5, 6, and 7. Execute each SELECT separately and compare the result grid to the values displayed in this report.